

Deploy da API em Ambiente de Produção

Publicar uma API não significa apenas colocá-la em execução em um servidor. O processo de deploy envolve preparação, isolamento de configurações, segurança, monitoramento e estabilidade operacional.

Objetivos de Aprendizagem

Ao final deste material, você será capaz de dominar os principais aspectos do deploy de APIs em ambiente de produção:

1

Desenvolvimento vs. Produção

Compreender as diferenças fundamentais entre os dois ambientes.

2

Preparar o NestJS

Configurar a aplicação para publicação segura e eficiente.

3

Variáveis de Ambiente

Utilizar variáveis de ambiente de forma adequada e segura.

4

Docker para Produção

Criar imagens e publicar containers de forma segura e estável.

O que Muda em Produção?

Ambiente de Desenvolvimento

Recursos voltados à produtividade do desenvolvedor:

- Hot Reload ativo
- Logs detalhados
- Swagger aberto
- Mensagens completas de erro
- Banco de dados local

Ambiente de Produção

As prioridades mudam completamente para atender usuários reais:

- **Segurança** – proteção de dados e acessos
- **Performance** – resposta rápida e eficiente
- **Estabilidade** – sem falhas inesperadas
- **Disponibilidade** – serviço sempre acessível
- **Monitoramento** – visibilidade do sistema

O ambiente precisa ser preparado para usuários reais. Nunca publique em modo desenvolvimento!

Build e Execução em Produção

Evite executar `npm run start:dev` em produção. Esse comando foi criado exclusivamente para desenvolvimento local. Siga o fluxo correto:



A pasta `dist/` contém o código JavaScript compilado e otimizado para execução em produção.

Variáveis de Ambiente e ConfigModule

Separando Configurações do Código

Valores sensíveis **nunca** devem ficar no código-fonte. Evite:

```
const jwtSecret = '123456';  
const dbPassword = 'senha123';
```

Utilize um arquivo `.env` com as configurações:

```
PORT=3000  
DATABASE_HOST=db  
DATABASE_PORT=5432  
DATABASE_NAME=app  
DATABASE_USER=postgres  
DATABASE_PASSWORD=postgres  
JWT_SECRET=minha-chave-secreta
```

Instalando e Configurando o ConfigModule

```
npm install @nestjs/config
```

No módulo principal:

```
import { ConfigModule } from '@nestjs/config';

ConfigModule.forRoot({
  isGlobal: true,
});
```

Obtendo configurações via injeção de dependência:

```
constructor(
  private configService: ConfigService
) {}

const secret = this.configService.get('JWT_SECRET');
```

A aplicação deixa de depender de valores fixos no código!

Criando o Dockerfile de Produção

O Dockerfile define como a imagem da aplicação será construída. Esse container executará a versão compilada da aplicação NestJS:

```
FROM node:22-alpine

WORKDIR /app

COPY package*.json ./

RUN npm install

COPY . .

RUN npm run build

CMD ["npm", "run", "start:prod"]
```

Gerando a Imagem

```
docker build -t users-api .
```

```
docker images
```

Executando o Container

```
docker run -d \  
-p 3000:3000 \  
--name users-api \  
users-api
```

A API estará disponível externamente na porta 3000.

Docker Compose e Reinicialização Automática

O Docker Compose facilita o gerenciamento de múltiplos serviços. Containers podem falhar – configure a reinicialização automática para garantir disponibilidade:

docker-compose.yml

```
services:  
  
api:  
  build: .  
  ports:  
    - "3000:3000"  
  restart: unless-stopped  
  
db:  
  image: postgres:17  
  restart: unless-stopped
```

Verificando Logs

Logs são essenciais após a publicação para monitorar o comportamento da aplicação:

```
docker logs users-api
```

Acompanhamento contínuo em tempo real:

```
docker logs -f users-api
```

A opção `restart: unless-stopped` melhora significativamente a disponibilidade da aplicação.

Segurança em Produção



Proteger o .env

Nunca envie o arquivo .env para o GitHub.
Adicione ao .gitignore:

```
.env
```



Ocultar Erros Internos

Em produção, evite expor detalhes técnicos ao usuário. Nunca retorne stack traces completos na resposta da API.



Controlar o Swagger

Muitas organizações optam por não expor documentação publicamente:

```
if (process.env.NODE_ENV
  !== 'production') {
  SwaggerModule.setup(
    'api', app, document
  );
}
```



Configurar CORS

Evite liberar acesso para qualquer origem.
Especifique os domínios permitidos:

```
app.enableCors({
  origin: [
    'https://app.empresa.com'
  ]
});
```

Defina o `NODE_ENV` corretamente: `NODE_ENV=development` para desenvolvimento e `NODE_ENV=production` para produção. Isso permite comportamentos diferentes conforme o ambiente.

Checklist Antes do Deploy

Antes de publicar sua API em produção, verifique cada item desta lista:

Aplicação

- ☒ Build executando sem erros
- ☐ Variáveis de ambiente configuradas
- ☐ Banco de dados acessível
- ☐ JWT funcionando corretamente
- ☐ Testes executados com sucesso

Infraestrutura & Segurança

- ☐ Swagger revisado (desabilitado em prod)
- ☐ Docker e imagem funcionando
- ☐ CORS revisado e configurado
- ☐ Arquivo .env no .gitignore
- ☐ Logs acessíveis e monitorados

Com todos os itens verificados, sua API está pronta para atender usuários reais com segurança e estabilidade!

Estrutura Recomendada e Conclusão

Organização do Projeto

Esta estrutura facilita manutenção e implantação:

```
project/  
├── src/  
├── test/  
├── Dockerfile  
├── docker-compose.yml  
├── .env  
├── .env.example  
└── package.json
```

Mantenha um arquivo `.env.example` com as variáveis necessárias (sem valores reais) para facilitar a configuração em novos ambientes.

Conclusão

Ao utilizar variáveis de ambiente, containers Docker e configurações específicas para produção, garantimos que a aplicação esteja preparada para atender usuários reais de forma:



Segura



Escalável



Previsível

